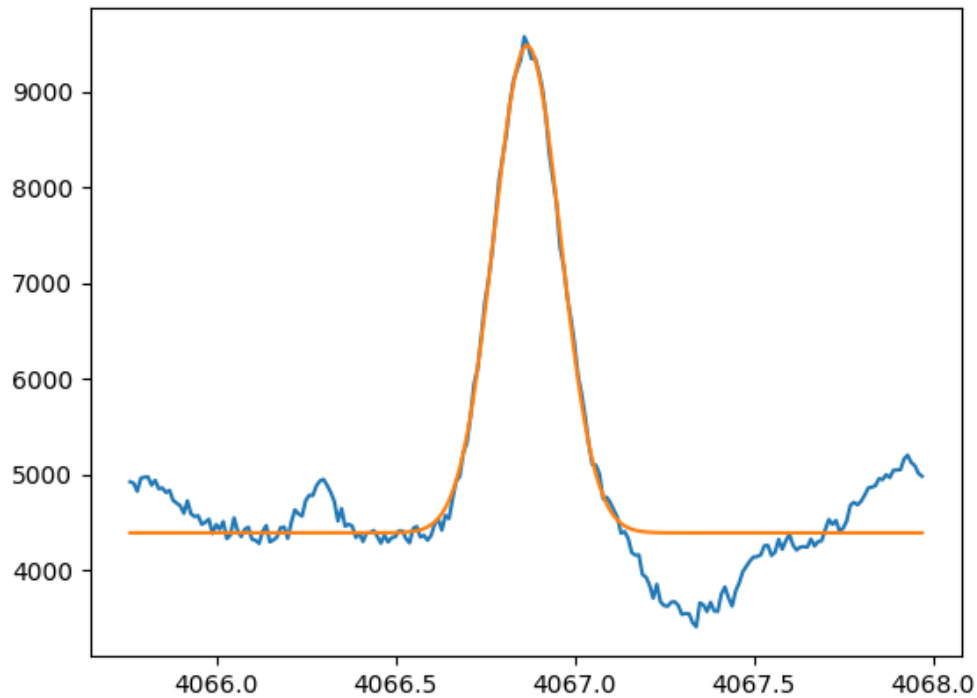




0.21965970150088393

Now for all ten stars, three amplitudes, and three fwhms:

```
[114]: # Injection test with Gaussian fitting. Inject n_injected gaussians into each
      ↪ of the observations, and count the percentage of spikes successfully
      ↪ recovered.
      # Do this for several areas[] under the Gaussian curve, and for several fwhms[].
      ↪ Fit a Gaussian to each detected signal,
      # and record the detected fwhm.

injection_tests = {'file': [], 'target': [], 'area': [], 'input_fwhm': [], 'detected':
      ↪ [], 'output_fwhm': []}
n_injected = 10
areas = [10000, 20000, 100000]
fwhms = [4, 10, 20]
for row in range(len(areas)):
    for col in range(len(fwhms)):
        print(areas[row], fwhms[col])
        area_injected = float(areas[row])
        fwhm_injected = float(fwhms[col])
```

```

for s in star_files:
    [wave,arr1] = optical_seti_functions.read_harps_file(s[0])
    if len(arr1)==0:
        continue
    # Perform SETI spike analysis on original spectrum
    hits_start_noinject, hits_end_noinject, count_noinject =
    ↪optical_seti_functions.seti_spike_analyzer(arr1, min_count = 4, max_count =
    ↪60, threshold_multiplier = 3.5, window_size = 1001, percentile=85)
    # Inject 10 random Gaussian signals
    for n in range(n_injected):
        center = random.randint(0,array_length)
        modded_spectrum = Gaussian_Injector.add_gaussian_to_array(arr1,
    ↪fwhm = fwhm_injected, amplitude=None, center=float(center),
    ↪array_length=array_length, axis=-1, area=area_injected)
        hits_start_inject, hits_end_inject, count_inject =
    ↪optical_seti_functions.seti_spike_analyzer(modded_spectrum, min_count = 4,
    ↪max_count = 60, threshold_multiplier = 3.5, window_size = 1001, percentile=85)
        # Keep track of only the new hits that *don't* occur in the
    ↪control run.
        hits_start = [x for x in hits_start_inject if x not in
    ↪hits_start_noinject]
        hits_end = [x for x in hits_end_inject if x not in
    ↪hits_end_noinject]
        # Add to output dictionary
        injection_tests['file'].append(s[0])
        injection_tests['target'].append(s[1])
        injection_tests['area'].append(areas[row])
        injection_tests['input_fwhm'].append(wave[center+int(fwhms[col]/
    ↪2)]-wave[center-int(fwhms[col]/2)])
        if len(hits_start) >= 1:
            if len(hits_start) > 1:
                print("Found more signals than I injected!")
                injection_tests['detected'].append(True)
                fwhm_recovered = optical_seti_functions.
    ↪gaussian_curve_fit(wave,modded_spectrum,hits_start[0],hits_end[0],plot=False)
                injection_tests['output_fwhm'].append(fwhm_recovered)
            else:
                injection_tests['detected'].append(False)
                injection_tests['output_fwhm'].append(0)
# Store results in an astropy table
t =
    ↪QTable([injection_tests['area'],injection_tests['input_fwhm'],injection_tests['output_fwhm']

```

10000 4

WARNING: AstropyDeprecationWarning: The Spectrum1D class is deprecated and may be removed in a future version.

Use Spectrum instead. [optical_seti_functions]

```
10000 10
```

```
WARNING: The fit may be unsuccessful; check:
```

```
The maximum number of function evaluations is exceeded.
```

```
[astropy.modeling.fitting]
```

```
10000 20
```

```
20000 4
```

```
20000 10
```

```
20000 20
```

```
Found more signals than I injected!
```

```
Found more signals than I injected!
```

```
Found more signals than I injected!
```

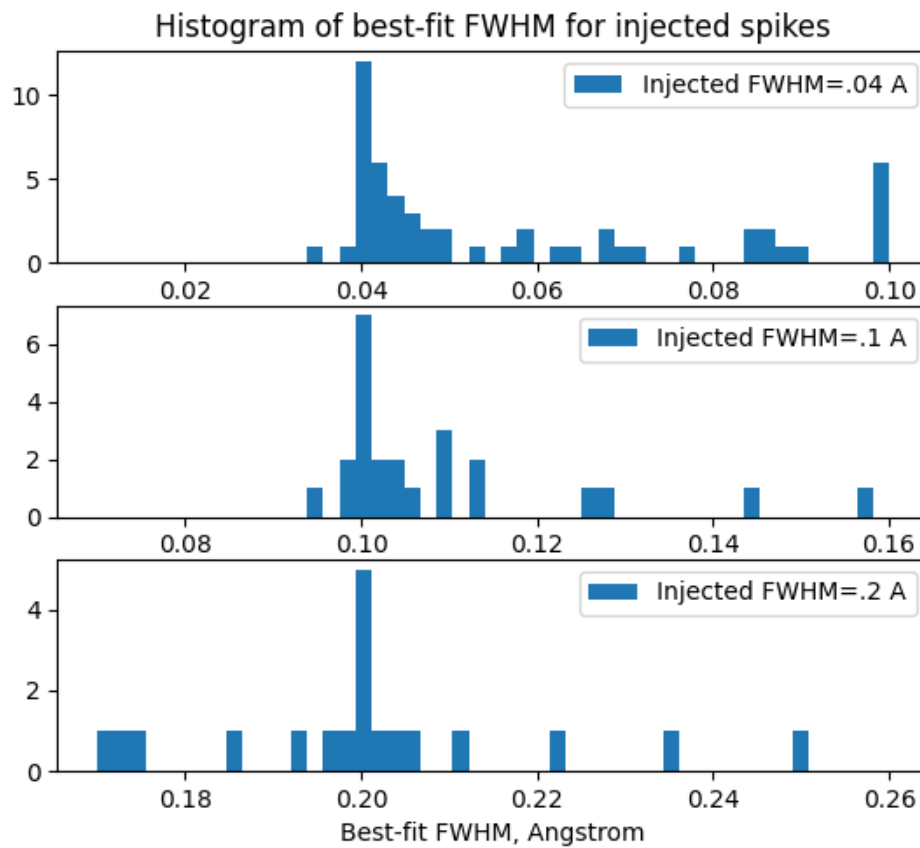
```
100000 4
```

```
100000 10
```

```
100000 20
```

```
[103]: mask = (t['input_fwhm'] < .05)
t_04 = t[mask]
mask = (t['input_fwhm'] > .05) & (t['input_fwhm'] < .12)
t_10 = t[mask]
mask = (t['input_fwhm'] > .12)
t_20 = t[mask]
```

```
[128]: plt.figure(5)
plt.clf()
bins = np.linspace(.01,.1,50)
plt.subplot(3,1,1)
plt.hist(t[(t['area'] == 10000)]['output_fwhm'],bins=bins)
plt.title('Histogram of best-fit FWHM for injected spikes')
plt.legend(['Injected FWHM=.04 A'])
plt.subplot(3,1,2)
plt.hist(t_10['output_fwhm'],bins=bins+.06)
plt.legend(['Injected FWHM=.1 A'])
plt.subplot(3,1,3)
plt.hist(t_20['output_fwhm'],bins=bins+.16)
plt.legend(['Injected FWHM=.2 A'])
plt.xlabel('Best-fit FWHM, Angstrom')
plt.show()
```



```
[130]: plt.savefig("gaussian-fit-injection.pdf",format="pdf")
```

```
[ ]:
```